

Introduction aux Graph Neural Networks (GNN) en Chimie

1. Le Problème des Réseaux Classiques

Un réseau de neurones classique perçoit une molécule uniquement à travers un vecteur de propriétés globales (masse molaire, logP, nombre d'atomes), appelé empreinte moléculaire (molecular fingerprint).

- **Exemple (Éthanol)** : La molécule est traduite par une série de descripteurs comme [46.07, 2, 1, 8, 0, -0.3, 20.2, 1, 1].
- **Limite majeure** : Deux molécules différentes peuvent partager des propriétés globales très similaires mais posséder des structures et des toxicités totalement distinctes. La connectivité et l'agencement des atomes sont perdus dans les descripteurs classiques.

L'approche GNN

Un GNN résout ce problème en représentant directement la structure chimique sous forme de graphe :

- Chaque atome devient un nœud (associé à ses caractéristiques : type, numéro atomique, charge, hybridation...).
- Chaque liaison devient une arête.

Analogie simple : Dans un réseau social, pour évaluer l'influence d'une personne, il ne suffit pas de regarder son profil isolé, il faut aussi analyser ses relations et son entourage (ses voisins). Le GNN fait exactement la même chose : chaque atome apprend en observant les atomes qui l'entourent.

2. Processus d'Apprentissage d'un GNN (Exemple de la Toxicité)

À partir d'une base de données de molécules connues (ex: Benzène → Toxique ; Éthanol → Non toxique), le réseau apprend de manière itérative en suivant 4 étapes :

Étape 1 : L'Observation Locale

Chaque atome examine ses caractéristiques propres et celles de ses voisins directs.

Exemple pour l'Oxygène dans une séquence C - O - H :

Nouvelle information de O = (w_O * O) + (w_C * C) + (w_H * H)

$$h_O^{(1)} = w_1 h_O^{(0)} + w_2 h_C^{(0)} + w_3 h_H^{(0)}$$

- $h^{(0)}$ représente l'information initiale brute de l'atome.
- $h^{(1)}$ information de l'atome après le 1^{er} passage.

Les coefficients (w) sont des poids ajustables que le réseau optimise pour identifier les informations utiles (ex: l'oxygène peut recevoir un poids plus fort si sa présence influence grandement la toxicité).

Étape 2 : La Propagation

Les informations circulent de proche en proche à travers le graphe.

- **1er passage** : L'atome découvre ses voisins immédiats (ex: l'oxygène connaît les caractéristiques du carbone adjacent).
- **2e passage** : L'atome intègre les informations des voisins de ses voisins (l'oxygène reçoit l'information du carbone situé plus loin).

$$h_O^{(2)} = w_1 h_O^{(1)} + w_2 h_C^{(1)} + w_3 h_H^{(1)}$$

- **Résultat** : À chaque étape, les informations sont mélangées à l'aide des poids. La signification chimique d'un atome dépend de son environnement.

Étape 3 : Création d'une Empreinte Moléculaire Apprise

Le GNN fusionne (par somme ou moyenne) les descriptions enrichies de tous les atomes pour générer un unique vecteur numérique : l'embedding moléculaire.

Contrairement aux descripteurs classiques, ces nombres n'ont pas de sens chimique direct (0.25 ne signifie pas 0.25 carbone). C'est une représentation interne optimisée par le réseau pour la tâche demandée.

Étape 4 : La Prédiction

Ce vecteur est envoyé à un petit réseau de neurones classique final qui calcule la probabilité du résultat.

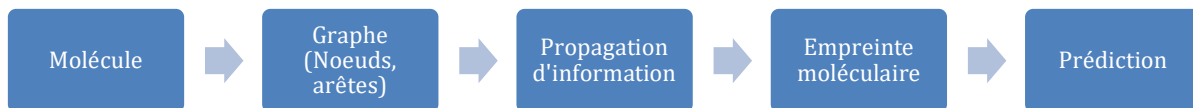
Toxicité = f(empreinte moléculaire) → 92% de probabilité (Toxique)

3. Exemples Concrets de Prédiction

Molécule	Structure	Détection par le GNN	Résultat
Molécule A : Éthanol	CH ₃ -CH ₂ -OH	Peu d'atomes lourds, pas de groupe fortement réactif.	Toxicité : 5 %
Molécule B : Chlorure de vinyle	CH ₂ = CH-Cl	Présence de chlore, structure associée à des composés cancérogènes connus.	Toxicité : 87 %

Conclusion

Les GNN sont particulièrement performants en chimie car ils respectent naturellement la structure géométrique et l'organisation des molécules. Ils automatisent à grande échelle le raisonnement logique du chimiste.



Annexes description des programmes et exercices proposés (notebook jupyter).

Tous les programmes sont téléchargeables sur le github suivant :

https://github.com/Didier06/Introduction_IA

1. IA_reseau_1neurone_TP_Chimie.ipynb

- **Fonction et Méthode** : Travail Pratique (TP) d'initiation conçu pour faire le pont entre la chimie et l'intelligence artificielle. Il utilise un réseau très basique composé d'**un seul neurone à activation linéaire** pour modéliser la **loi de Beer-Lambert** (Absorbance = epsilon * l * C).
- **Concepts abordés** : on y découvre de façon interactive l'impact des différents hyperparamètres (nombre d'epochs, choix de l'optimiseur entre SGD et Adam, influence du taux d'apprentissage ou "learning rate"). Dans une deuxième partie, l'étudiant(e) utilise ce neurone pour analyser un dosage expérimental réel de sulfate de cuivre. Dans ce cas pratique, le poids "W" appris par l'intelligence artificielle correspond en réalité au coefficient d'extinction molaire "epsilon", et le biais "b" correspond à l'absorbance résiduelle.

TP Chimie et IA : Initiation aux réseaux de neurones

Modélisation de la Loi de Beer-Lambert avec un seul neurone

Nom de l'étudiant : _____

Prénom de l'étudiant : _____

Date : 02 juin 2026

Rappel théorique pour le TP

En chimie, la **loi de Beer-Lambert** relie l'absorbance A à la concentration C :

$$A = \epsilon \cdot l \cdot C$$

En Intelligence Artificielle, un **neurone artificiel linéaire simple** calcule sa prédiction de sortie $\hat{y}$ en fonction de son entrée x par l'équation :

$$\hat{y} = W \cdot x + b$$

2. IA_reseau_2classes.ipynb

- **Fonction et Méthode** : Ce notebook est un cours interactif sur le problème de la **classification binaire (2 classes)**. Il génère d'abord un jeu de données de points en 2D (coordonnées x et y) appartenant à deux catégories distinctes.

- **Concepts abordés** : Le programme montre pas à pas comment un modèle linéaire basique échoue à séparer ces points, puis comment l'introduction d'un réseau avec une **fonction d'activation sigmoïde** permet de tracer une véritable "frontière de décision". Il permet de visualiser l'évolution des erreurs d'entraînement et conclut par une extension vers un problème à 3 classes en utilisant la fonction d'activation **Softmax**.

Contexte théorique du TP

En chimie médicinale, on cherche souvent à classer des molécules comme **actives** (Classe 1) ou **inactives** (Classe 0) selon leurs propriétés physico-chimiques. Dans ce TP, nous allons étudier deux descripteurs moléculaires clés :

1. Le **LogP** (Coefficient de partage octanol-eau), qui mesure le caractère hydrophobe de la molécule.
2. La **PSA (Polar Surface Area) divisée par 10**, qui mesure la surface polaire de la molécule (liée à sa capacité à former des liaisons hydrogène).

Contrairement au TP précédent (loi de Beer-Lambert) où la relation était purement continue et linéaire, nous voulons ici réaliser une **classification binaire** (séparer deux classes distinctes de points).

Le neurone de classification binaire :

Pour effectuer une classification binaire, un seul neurone calcule d'abord une combinaison linéaire de ses entrées x_1 (LogP) et x_2 (PSA/10) :

$$z = W_1 \cdot x_1 + W_2 \cdot x_2 + b$$

Pour convertir cette valeur continue z en une **probabilité** entre 0 et 1, on applique la fonction d'activation **Sigmoïde** (notée σ) :

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3. Toxicite_Keras_2.ipynb

- **Fonction et Méthode** : Ce fichier utilise un réseau de neurones dense (Multi-Layer Perceptron via Keras) pour prédire si une molécule est toxique ou non (classification binaire). La méthode s'appuie sur l'empreinte digitale de Morgan (Morgan Fingerprints), qui convertit la chaîne SMILES de la molécule en un vecteur binaire (2048 bits) représentant ses sous-structures locales avant l'entraînement.
- **Jeu de données** : Base de données *ClinTox*.
- **Nombre de molécules** : **1 480 molécules** valides (réparties en 1 184 pour l'entraînement et 296 pour le test).

Prédiction de Toxicité avec Vraies Données (ClinTox)

Nom :

Prénom :

Dans ce notebook, nous n'utilisons plus des données inventées. Nous allons télécharger la base de données **ClinTox** : un dataset scientifique qui contient des médicaments approuvés par la FDA et des molécules qui ont échoué lors d'essais cliniques en raison de leur toxicité.

```
1 # Installation de pandas si vous ne l'avez pas (retirer le # pour exécuter)
2 # %pip install pandas
```

✓ 0.0s

Python

```
1 import pandas as pd
```

4. Toxicite_GNN_DeepChem.ipynb

- **Fonction et Méthode** : Ce fichier remplace le réseau classique par un Réseau de Neurones en Graphes (GNN) à l'aide de DeepChem. Au lieu d'utiliser des vecteurs figés, le ConvMolFeaturizer convertit la molécule en un véritable graphe mathématique (nœuds = atomes, arêtes = liaisons), ce qui permet à l'IA de mieux appréhender la géométrie 3D pour détecter des toxicités plus subtiles (toujours en classification binaire).
- **Jeu de données** : Base de données *ClinTox*.
- **Nombre de molécules** : **~1 480 molécules** (1 184 pour l'entraînement, 148 pour le test, et le reste en validation).

GNN : Graph Neural Networks en Chimie avec DeepChem

Nom :

Prénom :

Faire un GNN "à la main" avec Keras est un cauchemar mathématique (car chaque molécule a un nombre d'atomes différent, ce que les réseaux classiques détestent).

Heureusement, la communauté scientifique a créé **DeepChem**, une bibliothèque open-source qui rend la création de GNN incroyablement simple !

```
1 # 1. Installation de DeepChem (retirer le # pour exécuter)
```

5. Toxicite_GNN_Tox21.ipynb

- **Fonction et Méthode** : Ce fichier utilise la même architecture de GNN (DeepChem et graphes moléculaires) mais s'attaque à un problème de prédiction "multi-tâches". Au lieu d'une toxicité binaire globale, l'algorithme prédit simultanément le risque de toxicité sur 12 cibles biologiques précises (ex: perturbation hormonale, dégâts sur l'ADN), offrant un profil toxicologique mécanistique complet.
- **Jeu de données** : Base de données *Tox21*.
- **Nombre de molécules** : **Plus de 8 000 molécules** (dont 6 258 pour l'entraînement et 783 pour le test).

GNN sur un dataset massif : Tox21

Nom :

Prénom :

Puisque **ClinTox** était trop petit (1500 molécules) pour que le GNN devienne vraiment intelligent, nous passons à la vitesse supérieure !

Nous allons utiliser **Tox21** (Toxicology in the 21st Century) : une initiative majeure du gouvernement américain. Ce dataset contient plus de **8 000 molécules**. Mieux encore : au lieu de prédire un simple "Toxique / Non Toxique", il prédit **12 cibles biologiques différentes** en même temps (ex: Est-ce que ça perturbe les hormones ? Est-ce que ça abîme l'ADN ?).

6. GNN_Toxicite_Ameliore.ipynb

- **Fonction et Méthode** : Ce fichier reprend le modèle multi-tâches (GNN sur 12 cibles) mais a pour fonction d'en améliorer les performances globales et la robustesse (score ROC-AUC). La méthode repose sur une optimisation fine des hyperparamètres : régularisation augmentée (dropout de 0.3), ajustement du taux d'apprentissage (5e-4), lots de 64, et un entraînement prolongé sur 50 itérations (epochs).
- **Jeu de données** : Base de données *Tox21*.
- **Nombre de molécules** : **Plus de 8 000 molécules** (identique au précédent, 6 258 en entraînement et 783 en test).

Partie 4 : Bilan sur l'amélioration du modèle

Questions :

1. **Hyperparamètres** : Dans ce modèle amélioré, nous avons ajusté le `dropout` à 0.3 et le `learning_rate` à $5e-4$. À quoi sert la technique du *dropout* pendant l'entraînement d'un réseau de neurones ?
2. **Durée de l'entraînement** : Le paramètre `nb_epoch` est passé de 10 à 50. Quel est l'avantage de laisser le modèle s'entraîner plus longtemps, et quel est le risque principal si on l'entraîne trop (surapprentissage ou overfitting) ?
3. **Score et limites** : Le score ROC-AUC s'est amélioré (environ 0.697) par rapport au modèle de base. Pourquoi est-il si difficile d'obtenir un modèle qui prédit à 100% la toxicité d'une molécule sur un organisme vivant ?

(Double-cliquez sur cette cellule pour rédiger vos réponses ci-dessous :)

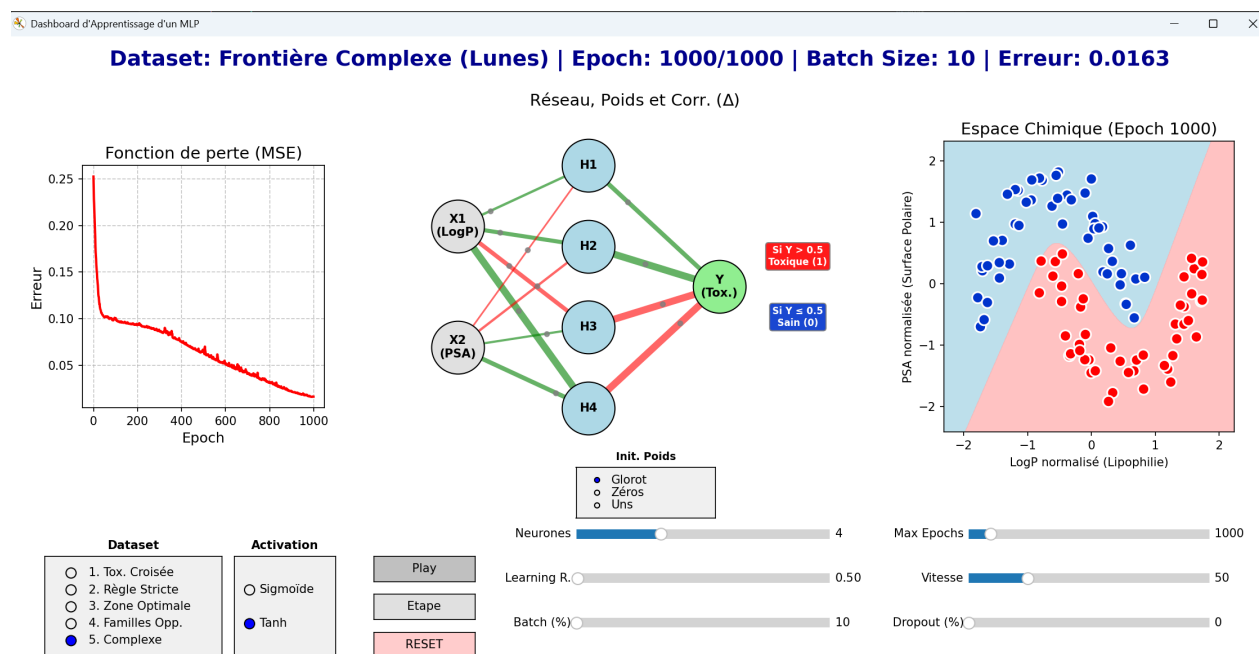
- Réponse 1 : ...
- Réponse 2 : ...
- Réponse 3 : ...

Annexe 2 programmes pédagogiques python

Descriptif des trois programmes pédagogiques Python présents dans le répertoire Chimie_IA :

1. mlp_pedagogique.py

Ce script propose un **tableau de bord interactif** (réalisé avec Matplotlib) permettant de visualiser en temps réel le fonctionnement d'un **Multi-Layer Perceptron (MLP)** classique. L'interface offre à l'utilisateur la possibilité de modifier des hyperparamètres (nombre de neurones, taux d'apprentissage, choix du jeu de données, etc.), de tester différentes fonctions d'activation (Sigmoid, Tanh) et de voir étape par étape comment le réseau s'entraîne et ajuste ses poids pour classer les données.



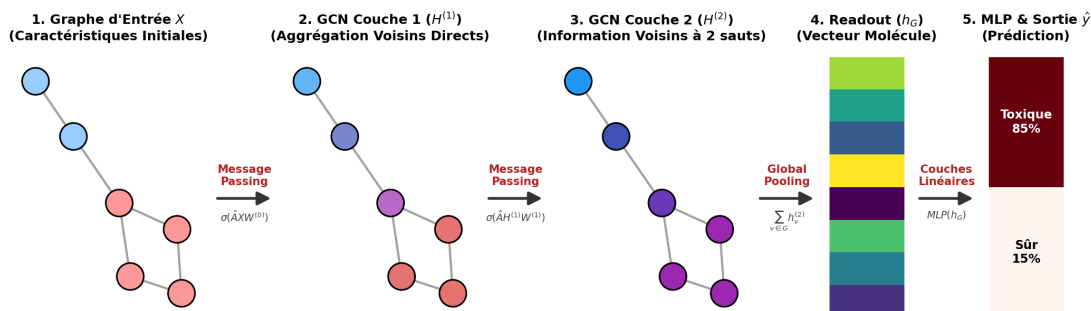
2. gnn_architecture.py

Ce programme est un **outil de visualisation graphique statique/interactif** (avec NetworkX) qui décompose l'architecture interne d'un **Graph Neural Network (GNN)**. Il explique visuellement, étape par étape, le cheminement de l'information : les caractéristiques initiales des atomes, le brassage des informations via le "Message Passing" (Couches GCN 1 et 2), l'agrégation finale de la molécule ("Global Pooling") et la classification via un MLP. Un bouton permet même d'afficher les dimensions mathématiques des matrices à chaque étape.

Afficher Dim.

Visualisation de l'Architecture d'un Graph Neural Network (GNN)

Détail de l'équation $\sigma(\hat{A}XW^{(0)})$: X = Infos atomes | $W^{(0)}$ = Poids (Transformation) | $\hat{A}$ = Liens (Mélange voisins) | σ = Activation

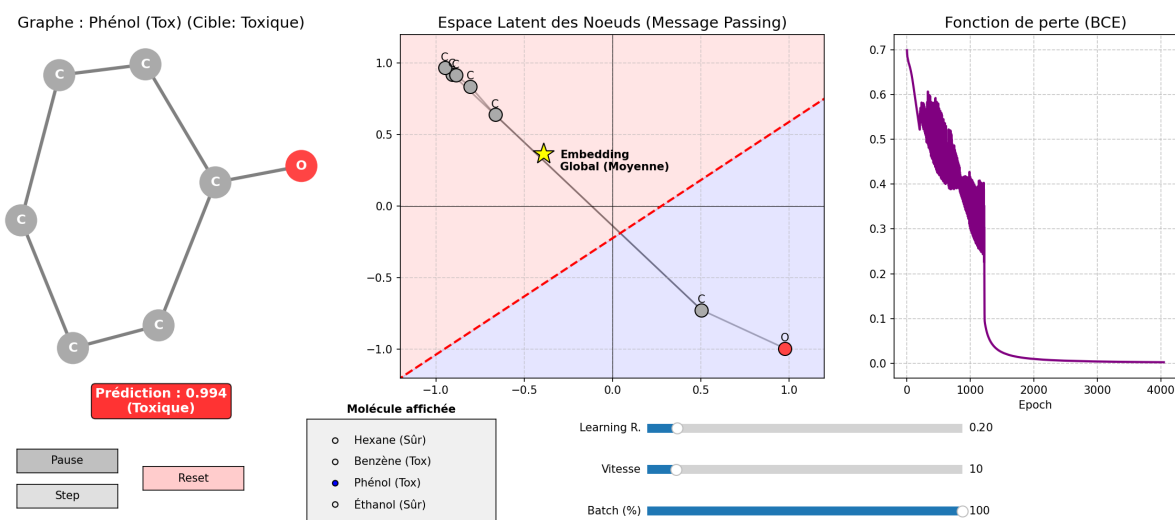


1. Chaque atome (noeud) commence avec ses propres caractéristiques (vecteur d'entrée X).
2. Couche GCN 1 : Chaque atome reçoit et agrège les informations de ses voisins directs (les couleurs se mélangent).
3. Couche GCN 2 : Un atome reçoit des informations des voisins de ses voisins (le contexte devient plus large).
4. Global Pooling : On combine les états finaux de tous les atomes (somme, moyenne) pour créer une seule 'empreinte' de la molécule entière.
5. MLP : Un réseau de neurones classique analyse cette empreinte moléculaire pour produire la prédiction finale (ex: Toxicité).

3. gnn_pedagogique.py

C'est un **tableau de bord dynamique et interactif** dédié à l'apprentissage des **GNN appliqués à la chimie**. Il modélise des molécules simples (Hexane, Benzène, Phénol, Éthanol) sous forme de graphes. L'utilisateur peut lancer l'entraînement en direct ("Play / Step") et observer trois éléments clés : la molécule et la prédiction de sa toxicité, la projection des atomes dans un **espace latent 2D** (montrant comment l'IA regroupe les caractéristiques et trace sa frontière de décision), ainsi que la courbe de l'erreur globale (Loss) qui diminue au fil des "epochs".

GNN Dashboard | Epoch: 4050 | Erreur: 0.0026



Annexe 3 l'assistant IA

Pour vous accompagner dans votre apprentissage de l'Intelligence Artificielle et des réseaux de neurones, un "Assistant IA et Python" basé sur Langflow est désormais directement intégré à votre plateforme Moodle.

Spécialisé sur l'IA les réseaux de neurones et python, ce tuteur virtuel interactif est conçu pour vous aider à résoudre vos travaux pratiques et à éclaircir les concepts complexes de Deep Learning.

Son onglet principal vous guide pas à pas pour que vous puissiez résoudre vos exercices, tandis que l'onglet "Entraîneur QCM" vous permet d'évaluer vos connaissances de base de manière ciblée.

Une interface de messagerie intuitive vous offre la possibilité de poser vos questions à tout moment, en y incluant même des formules mathématiques.

Attention toutefois à garder un esprit critique : en tant qu'intelligence artificielle, l'assistant peut parfois se tromper ou faire preuve d'imprécision. Utilisez-le comme une aide à la réflexion, et bonne découverte de l'IA !

Assistant IA et Python
Console Enseignant — Analyse et Supervision

DO Didier Orlandi

Assistant

Entraîneur QCM

Évaluateur Prof

Bonjour Didier !

Posez vos questions sur les réseaux de neurones à l'assistant IA et testez vos connaissances de base sur les réseaux de neurones et l'IA
Attention l'assistant peut être imprécis ou faire des erreurs.
Bonne découverte de l'IA et du Deep Learning !

Qu'est ce qu'un GNN ? En 5 lignes maximum.

DO

Voici une explication ultra-simple en 5 lignes, Didier ! 😊

1. Un **GNN** (*Graph Neural Network*) est un réseau de neurones conçu pour travailler avec des **données en forme de graphe** (nœuds + arêtes, comme un réseau social ou une carte routière).
2. Il **propage des informations** entre nœuds voisins (comme un jeu de téléphone) pour mettre à jour leurs états.
3. **Exemple** : Prédire le prix d'une maison en tenant compte de ses voisines (via leurs relations géographiques).
4. **Formule clé** : Chaque nœud calcule son nouvel état en agrégeant les messages de ses voisins :